

PIPELINE_RESILIENCE_SPEC_v1.md

Runtime implementation of Doctrine #31 (Infrastructure Resilience & Jurisdictional Independence)

Status: Draft v1 — pending Layer 4 consent review **Owner:** QashqAI Voice / Siefollah Aslani **Date:** 2026-06-17 **Scope:** Production runtime of qashqaivoice.com only (three-agent Claude API pipeline). Does NOT touch raw-data path (already protected by Doctrine #28/#29) or offline masters (Doctrine #31 asset sovereignty).

1. Why this exists (the trigger)

As of mid-June 2026, Anthropic infrastructure showed a clear pattern: ten significant service disruptions in twelve days (June 5–16), with Anthropic publicly acknowledging that demand has outpaced infrastructure capacity, especially at peak hours. Combined with (a) export-control disabling of the Fable 5 / Mythos 5 tier and (b) consumer-tier identity verification, single-vendor dependency has moved from a theoretical watchlist item to a realized systemic risk.

This spec converts that risk from *unmanaged* to *managed*. It does not remove dependency on Claude — it makes the dependency a **choice**, not a lock-in.

2. The exposure, by layer

Layer	Component	Outage impact	Cultural-asset risk
Build	Claude Code (Layer 1 Builder)	Dev velocity stalls; async, deferrable	None
Runtime	Claude API — Language Detector → Cultural Guardian → Reasoning	Live platform features degrade/fail	Indirect (see §4)
Data	Raw recordings (father / Layer 4)	None — no LLM in raw-data runtime path	None — offline masters exist

The only user-facing exposure is the runtime pipeline. This spec addresses it.

3. Core principle — FAIL CLOSED on cultural safety

Asymmetric reversibility applied to runtime: an outage must never cause a cultural-safety **bypass**. A degraded or refused response is reversible; an extractive or non-consented response that has already reached a user is not.

Therefore the **Cultural Guardian gate is never skipped, never failed-open**. If the Guardian cannot run, the system degrades to a non-generative tier (§5, Tier 2). This is the runtime expression of the Cultural Asset Priority Arbitration Rule already in Doctrine #31.

4. Data-sovereignty constraint on ANY failover

A naive “add a second LLM” failover violates the Non-Extractive Data Sovereignty non-negotiable, because it would route cultural content to a second foreign vendor.

Rule: A fallback LLM provider may process cultural input ONLY if it operates under contractual terms equivalent to the current standard — no training on inputs, zero/limited retention, no advertising/marketing use, EU-eligible processing. If no compliant provider is available at outage time, the pipeline does NOT route cultural data anywhere; it goes to Tier 2.

This means the failover decision is **governance-gated, not latency-gated**.

5. Three-tier graceful degradation model

Tier 0 — Normal

Full Claude pipeline: Language Detector → Cultural Guardian → Reasoning Agent (Claude API).

Tier 1 — Claude degraded / unavailable

- **Language Detector → local deterministic fallback.** Language detection across Persian/Qashqai (Perso-Arabic script) vs. German/English (Latin) does not require an LLM. Use Unicode-range heuristics + a local library (e.g. a self-hosted `fastText/ langdetect` -class classifier) inside QashqAI Voice-controlled European infrastructure. *This removes Layer A from Claude dependency permanently — implement regardless of outage state.*
- **Cultural Guardian → must still run.** If a compliant provider (§4) is reachable, route

the Guardian gate to it. If not → escalate to Tier 2. Never auto-pass.

- **Reasoning Agent → compliant-provider failover** (e.g. GPT-5.5 / Layer 2 infrastructure) ONLY after Guardian passes AND only under §4 terms. Tag responses generated in Tier 1 with an internal provenance flag for audit.

Tier 2 — No compliant generative path available

- **Fail closed to deterministic service.** Serve from the verified Living Dictionary (363+ entries) via direct lookup — no LLM, fully sovereign, already consented.
- Display a transparent, multilingual “service temporarily degraded” notice (Persian / Qashqai / German / English). Honesty over silent failure — consistent with project transparency ethic.
- Queue any unservable generative requests for later; never silently route cultural data to an unvetted provider.

6. Implementation checklist (for Layer 1 / Claude Code)

- Add a local language-detection module (Unicode-range + local classifier); make it the default for Layer A, with Claude as optional enrichment only.
- Wrap each agent call in a circuit breaker with explicit tier-fallback routing (not blind retry).
- Add a provider-abstraction interface so Reasoning/Guardian providers are swappable via config, not code (Doctrine #31 substitutability).
- Implement a `complianceCheck(provider)` guard that blocks any fallback provider lacking §4 terms.
- Build the Tier 2 Living-Dictionary deterministic responder + multilingual degraded-service notice.
- Add provenance tagging: every response records which tier/provider produced it (audit + Layer 4 transparency).
- Add a status/health endpoint and structured logging of tier transitions.

7. Governance & consent

- This spec changes the runtime path; per project governance it requires **Layer 4 (cultural validation) consent review** before merge — same gate as PR #12.

- The Tier 1 fallback provider's data terms must be reviewed and recorded in Evidence & Sources before activation. No provider goes live on cultural data without that record.

8. Open decisions (Siefollah only)

1. **Fallback provider for cultural data:** confirm whether the Layer 2 (GPT-5.5) API contract meets §4 terms (no-training + retention). If yes → eligible for Tier 1. If unverified → Reasoning Agent has NO LLM failover and Tier 1 collapses into Tier 2 for generation (Language Detector still gets its local fallback). Decision gates the whole §5 Tier 1.
2. **Tier 2 scope:** Living Dictionary only, or also a small set of pre-approved static responses?
3. **Sequencing:** ship the local Language Detector first (pure win, no governance gate), or design the full circuit-breaker layer in one slice?